

10c — Sharding Strategies

Key Concepts

Term	Definition
Sharding	Splitting data horizontally across multiple DB servers
Shard	One partition/slice of the total dataset
Shard Key	The column used to decide which shard a row belongs to
Hotspot	One shard receiving disproportionately more traffic than others
Rebalancing	Redistributing data when shards are added or removed
Cross-shard Query	A query that must touch more than one shard — expensive

Replication vs Sharding

	Replication	Sharding
Data	Same data on all servers	Different data on each server
Solves	Read scale + availability	Write scale + storage scale
When	Read-heavy, need failover	Single server maxed out entirely

How It Works

Range-based:

```
Shard 1: user_id 1 - 1,000,000
Shard 2: user_id 1,000,001 - 2,000,000
Shard 3: user_id 2,000,001 - 3,000,000
```

Hash-based:

```
shard = hash(user_id) % number_of_shards
→ Even distribution, but range queries span all shards
```

Directory-based:

```
Lookup table: user_id 101 → Shard 2
→ Flexible but lookup table is a single point of failure
```

Sharding Strategies Compared

Strategy	Pro	Con
Range-based	Simple, range queries fast	Hotspots (new data always hits last shard)
Hash-based	Even distribution, no hotspots	Range queries expensive, rebalancing painful
Directory-based	Flexible, easy to move data	Lookup table = bottleneck + single point of failure

Choosing a Shard Key

Good shard key: High cardinality, evenly distributed, frequently used in queries

- `user_id, order_id` ✓

Bad shard key examples:

Bad Key	Problem
<code>created_at</code>	All new data hits one shard — hotspot
<code>country</code>	USA gets 10x traffic of other shards
<code>status</code>	Only 3-4 values — massively uneven

Hard Problems with Sharding

1. **Cross-shard JOINS** — expensive or impossible; data is on different servers
2. **Cross-shard transactions** — ACID guarantees break across shard boundaries
3. **Rebalancing** — adding a shard means redistributing data (especially painful with hash sharding)
4. **Operational complexity** — managing N databases instead of one

Sharding is a last resort — exhaust vertical scaling, indexing, caching, and replication first.

Industry Examples

- **WhatsApp** — shards by `chat_id`; all messages in a conversation stay on one shard
 - **Uber** — shards by `city_id`; all London trips on one shard, NYC on another
 - **Discord** — moved to Cassandra (built-in sharding) when single Postgres hit limits at billions of messages
 - **Instagram** — shards by `user_id`; all photos for a user stay on the same shard
-

Interview Questions

Q: What is sharding and how is it different from replication? A: Sharding splits data across servers (different data per server) for write/storage scale. Replication copies data across servers (same data) for read scale and availability.

Q: What is a shard key and why does it matter? A: The column used to route rows to shards. A bad shard key causes hotspots or uneven distribution — one of the most critical decisions in sharding.

Q: What are the downsides of sharding? A: Cross-shard joins are hard, ACID transactions break across shards, rebalancing is painful, and operational complexity increases significantly.

Q: Hash vs range sharding — when do you choose each? A: Hash for even write distribution when range queries aren't needed. Range when range queries are common and hotspots can be managed.

Q: Why is sharding considered a last resort? A: It introduces massive complexity. Most scale problems can be solved first with indexing, caching, replication, or vertical scaling.